

Object Slicing in C++

Object slicing happens when a **derived-class object** is copied into a **base-class object** by value.
The derived-specific part gets "**sliced off**."

1. The basic example

```
class Animal {
public:
    int age = 5;
};

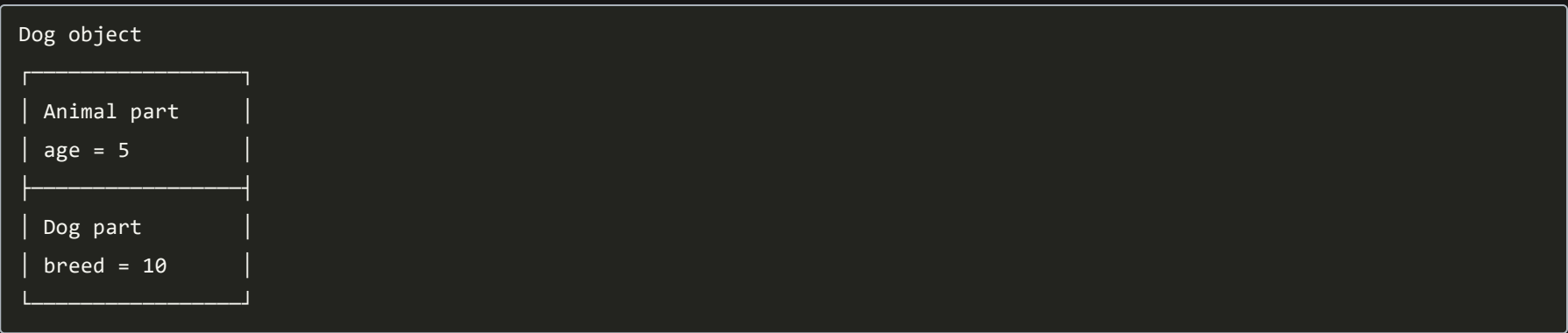
class Dog : public Animal {
public:
    int breed = 10;
};
```

Now:

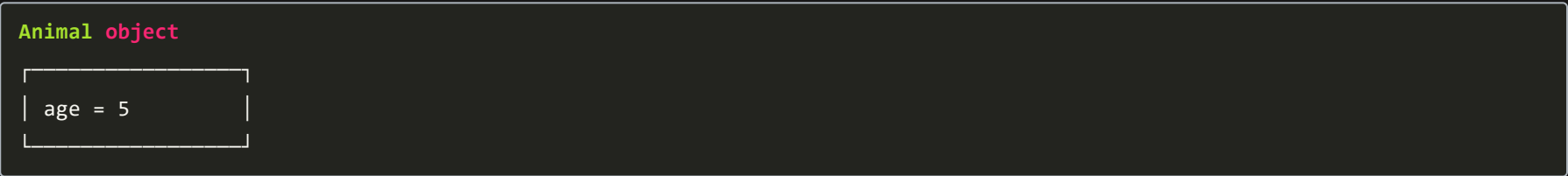
```
Dog d;

Animal a = d;    // Object slicing
```

d contains:



But **a** is an actual **Animal object**:



The **Dog** portion is sliced away.

2. Why is this a problem?

The biggest problem appears with **polymorphism**.
Consider:

```
class Animal {
public:
    virtual void speak() {
        cout << "Animal";
    }
};

class Dog : public Animal {
public:
    void speak() override {
        cout << "Dog";
    }
};
```

Now:

```
Dog d;

Animal a = d;

a.speak();
```

Output:

```
Animal
```

Why?

Because:

```
Animal a = d;
```

created a **new Animal object**.

The **Dog** portion no longer exists inside **a**.

So even though **speak()** is virtual, there is no longer a **Dog** object to dispatch to.

3. How to avoid object slicing?

Use a reference

Instead of:

```
Animal a = d;    // slicing
```

use:

```
Animal& a = d;
```

Now no new object is created.

a simply refers to the existing **Dog**.

```
a.speak();
```

Output:

```
Dog
```

Because **a** refers to an actual **Dog** object and **speak()** is virtual.

Or use a pointer

```
Animal* a = &d;
```

```
a->speak();
```

Output:

```
Dog
```

Again, no slicing occurs.

4. The most important distinction

✗ By value

```
Animal a = dog;
```

Slicing occurs.

☑ By reference

```
Animal& a = dog;
```

No slicing.

☑ By pointer

```
Animal* a = &dog;
```

No slicing.

This is one of the most important things to remember.

5. Function parameters — common interview trap

Suppose:

```
void print(Animal a) {  
    a.speak();  
}
```

Calling:

```
Dog d;  
print(d);
```

causes **slicing**.

Because `d` is copied into an `Animal` parameter.

Better:

```
void print(const Animal& a) {  
    a.speak();  
}
```

Now:

```
Dog d;  
print(d);
```

No slicing.

And because `speak()` is virtual:

```
Dog
```

is printed.

★ Interview rule

When accepting polymorphic objects, prefer:

```
const Base&
```

or:

```
Base*
```

rather than:

```
Base
```

6. What about returning objects?

This can also cause slicing.

```
Dog d;  
  
Animal getAnimal() {  
    return d;  
}
```

The return type is `Animal`, so the `Dog` part is sliced when returned by value.

If polymorphism is intended, use a suitable reference/pointer/ownership mechanism instead, such as:

```
Animal* getAnimal() {  
    return new Dog();  
}
```

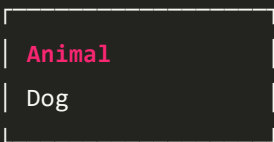
Though in modern C++, prefer smart pointers:

```
unique_ptr<Animal> getAnimal() {  
    return make_unique<Dog>();  
}
```

7. Object slicing vs pointer/reference

Think about it like this:

```
Dog d
```



By value:

```
Animal a = d;
```

creates:

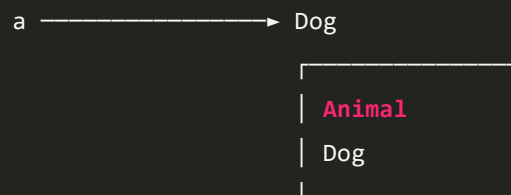


Dog part gone → slicing

Reference:

```
Animal& a = d;
```

creates no new object:



No slicing

Interview Questions

Q1. What is object slicing?

Object slicing occurs when a derived-class object is copied or assigned to a base-class object by value, causing the derived-specific members to be lost.

Q2. How do you prevent object slicing?

Use:

```
Base&
```

or:

```
Base*
```

instead of:

```
Base
```

for polymorphic objects.

Q3. Does slicing happen with references?

No.

```
Base& b = derived;
```

No object is copied.

Q4. Does slicing happen with pointers?

No.

```
Base* b = &derived;
```

The pointer just points to the existing object.

Q5. Does a virtual function prevent slicing?

No.

This is a common trick question.

```
Base b = Derived();  
b.foo();
```

Even if `foo()` is virtual, slicing has **already happened**.

Virtual dispatch can't recover the sliced-off `Derived` portion.

One-line interview memory

Polymorphism + inheritance → avoid passing/storing objects by Base value; use Base reference or pointer.

And remember:

```
Base b = derived;           // ✗ slicing
Base& b = derived;         // ✓
Base* b = &derived;        // ✓
```

This topic connects directly to the next major interview concept: **virtual functions + virtual destructors + polymorphism**, because object slicing is one of the reasons we use base-class pointers/references in the first place.